CSCI 3060U Phase 3: AI Use Acknowledgement

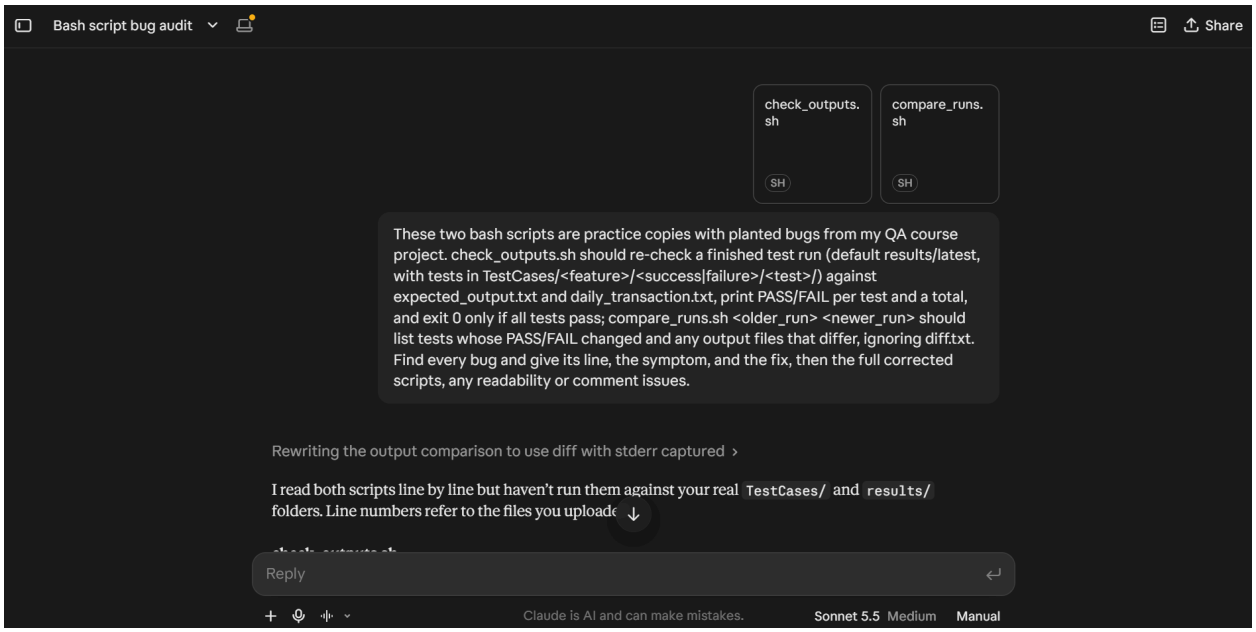
Team Members: Evan Harder, Muhammad Niazi, Maheen Zahir

Generative AI was used in this project phase. As required by the Phase 3 handout, the complete responses for AI conversation used on this phase is attached after this page, unedited and in full.

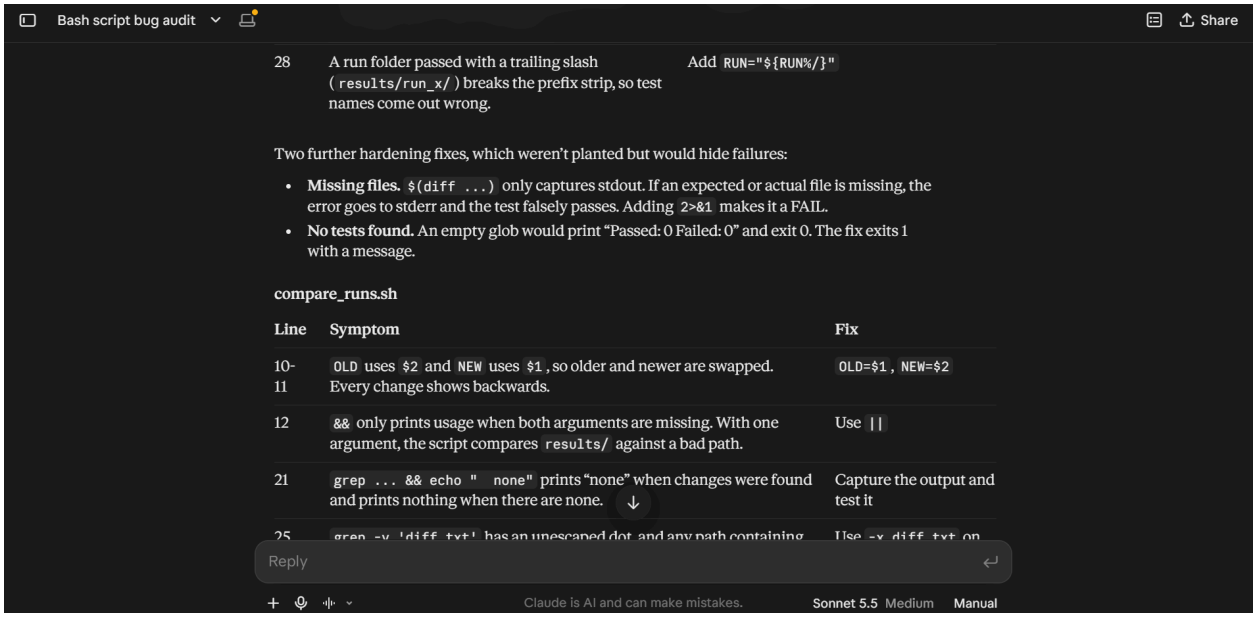
What the AI was used for:

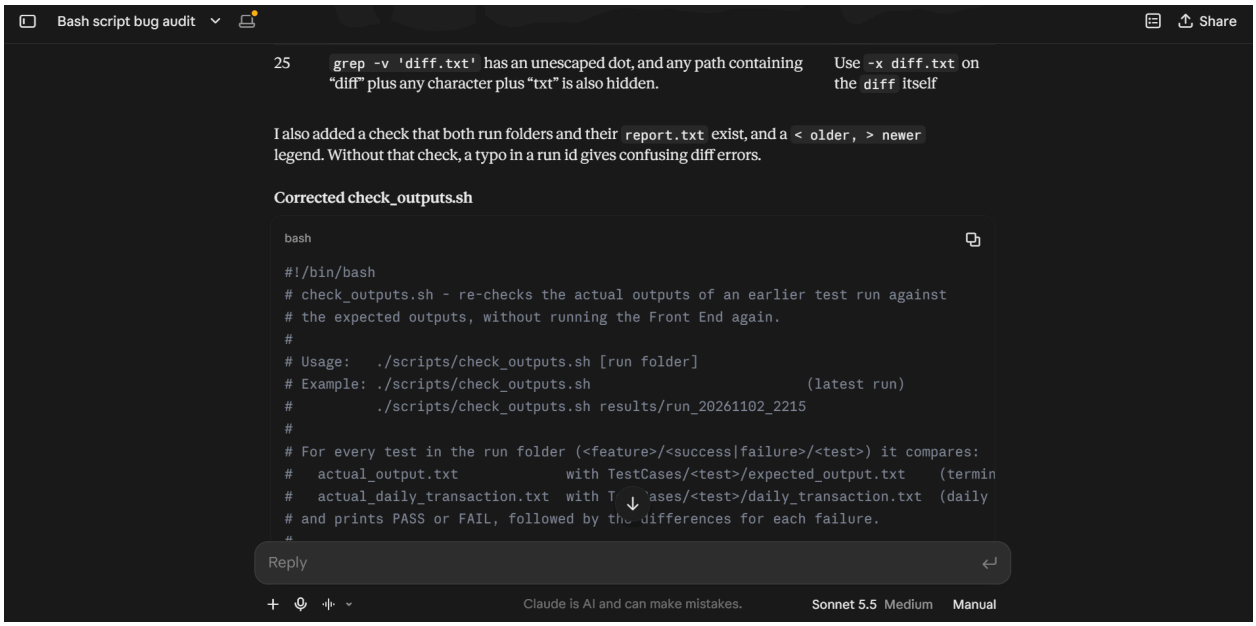
- Conversation 1: Asked to find and fix the planted bugs in two practice Bash scripts (check_outputs.sh and compare_runs.sh) and to suggest readability and comment improvements.

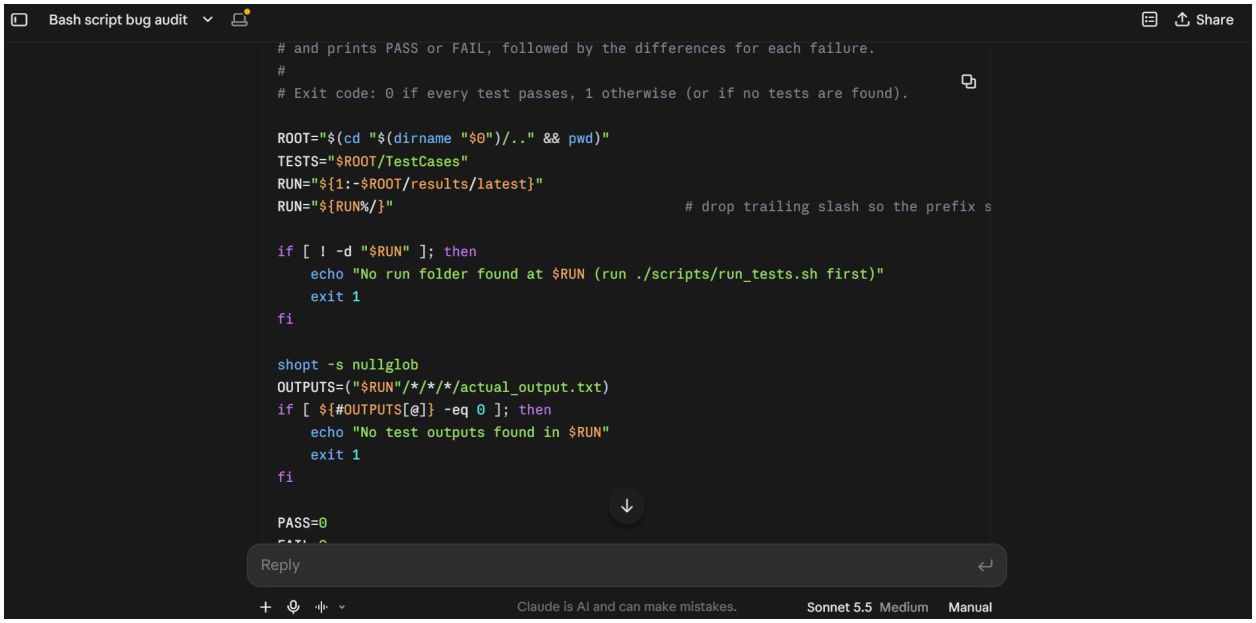
Conversation 1 image(s):

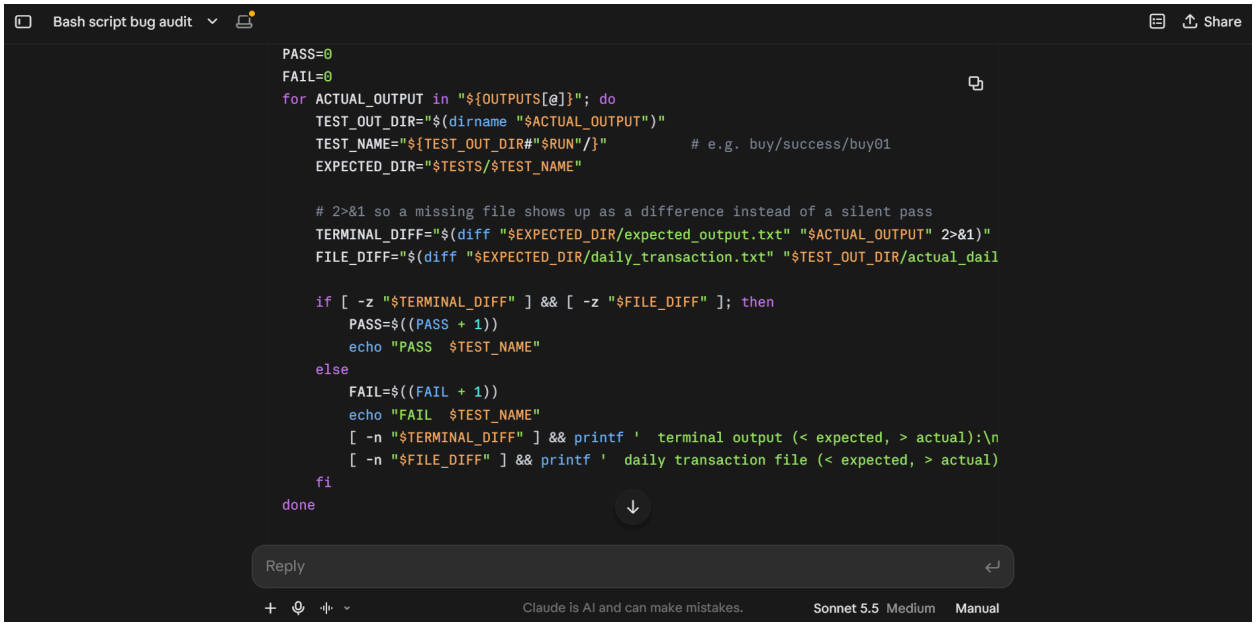


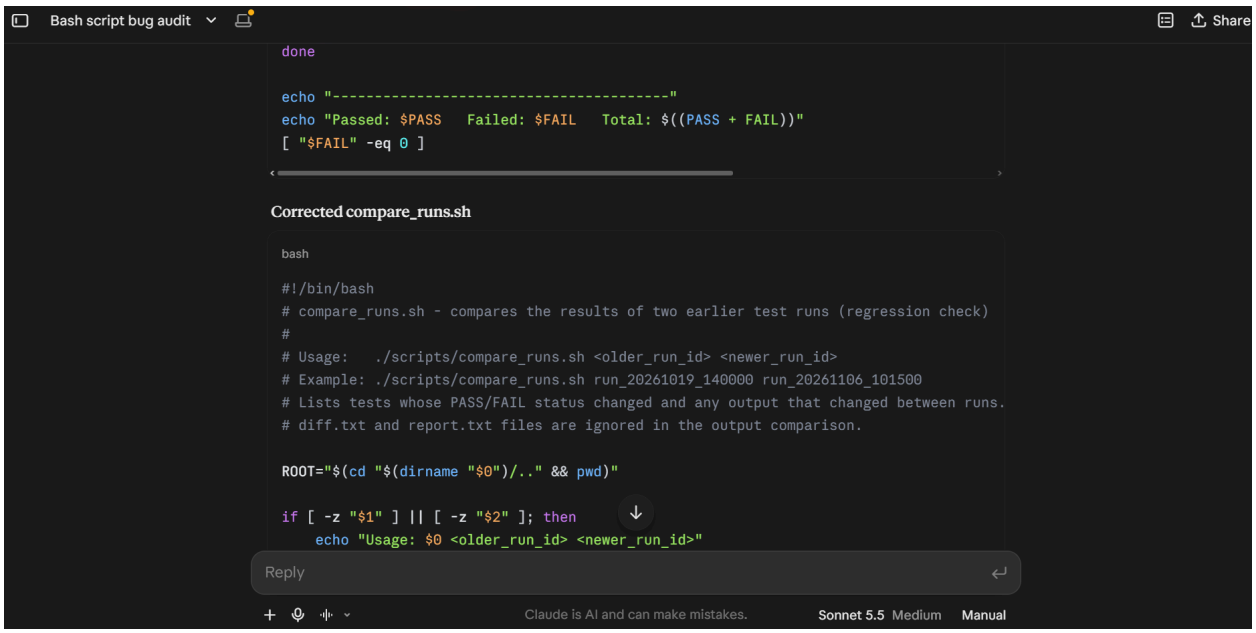

Bash script bug audit
Share
check_outputs.sh
Line
Symptom
Fix
17
The default is result/latest, so running with no argument always prints "No run folder found". The folder is results.
RUN="${1:-$ROOT/results/latest}"
26
The glob */*/ only goes two levels deep, but tests live at feature/success/test. It never finds actual_output.txt, so no real test is checked.
"$RUN"/*/*/*/actual_output.txt
32
The daily transaction file is compared against expected_output.txt, the terminal log. Every test whose terminal log and transaction file differ gets a bogus diff.
Compare against $EXPECTED_DIR/daily_transaction.txt
34
|| marks a test PASS if either diff is empty, so a test with one wrong output still passes.
Use &&
38
FAIL=$((PASS + 1)) makes the failure count wrong.
FAIL=$((FAIL + 1))
47
[$FAIL -ne 0] is inverted: it exits 0 whe
["$FAIL" -eq 0]
tests failed and 1 when all passed.
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual

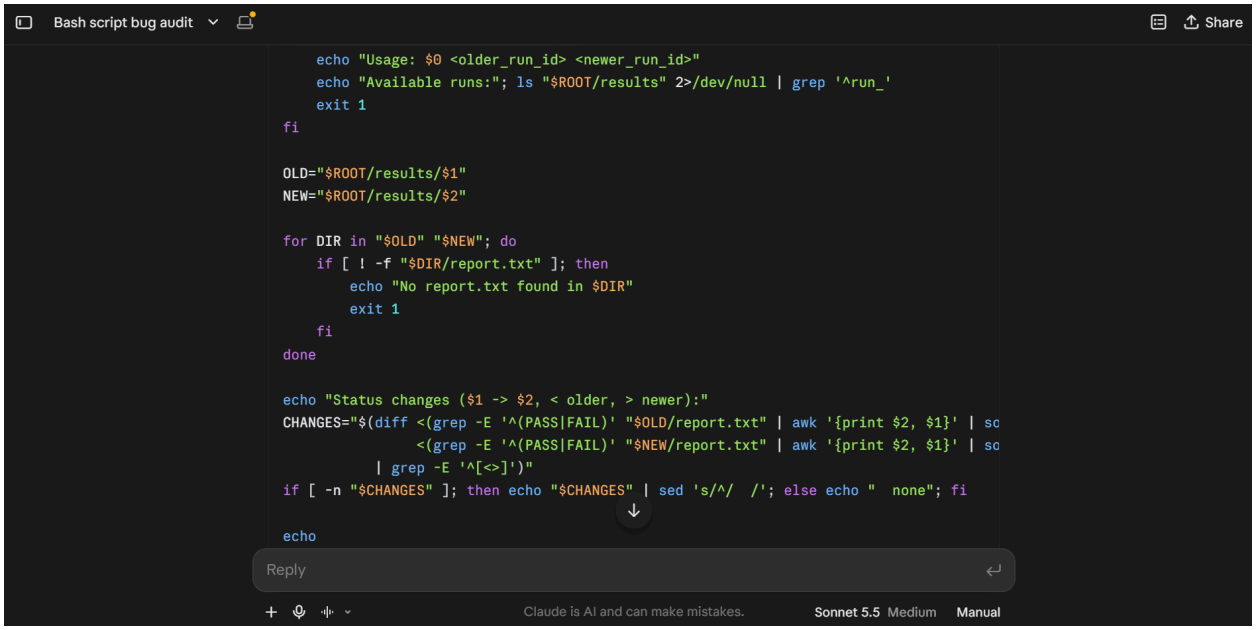

Bash script bug audit
Share
28
A run folder passed with a trailing slash (results/run_x/) breaks the prefix strip, so test names come out wrong.
Add RUN="${RUN%/}"
Two further hardening fixes, which weren't planted but would hide failures:
Missing files. $(diff ...) only captures stdout. If an expected or actual file is missing, the error goes to stderr and the test falsely passes. Adding 2>&1 makes it a FAIL.
No tests found. An empty glob would print "Passed: 0 Failed: 0" and exit 0. The fix exits 1 with a message.
compare_runs.sh
Line
Symptom
Fix
10-11
OLD uses $2 and NEW uses $1, so older and newer are swapped. Every change shows backwards.
OLD=$1, NEW=$2
12
&& only prints usage when both arguments are missing. With one argument, the script compares results/ against a bad path.
Use ||
21
grep ... && echo " none" prints "none" when changes were found and prints nothing when there are none.
Capture the output and test it
25
grep -v 'diff.txt' has an unescaped dot, and any path containing
Use -x diff.txt on
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual

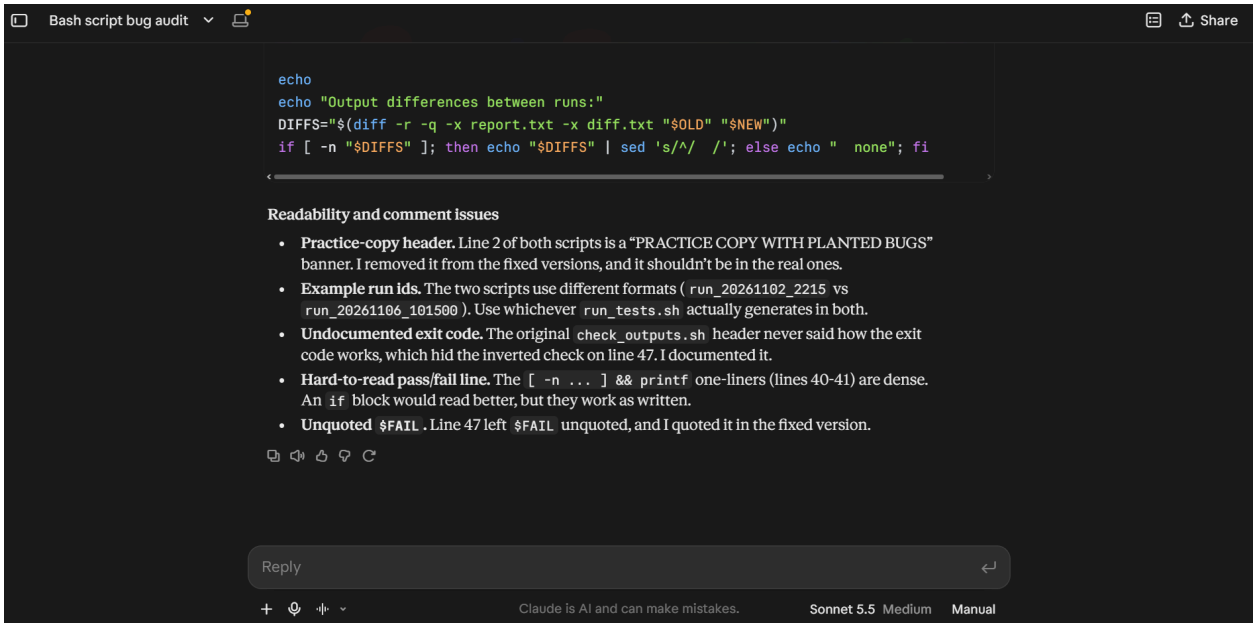

Bash script bug audit
Share
25 grep -v 'diff.txt' has an unescaped dot, and any path containing "diff" plus any character plus "txt" is also hidden. Use -x diff.txt on the diff itself
I also added a check that both run folders and their report.txt exist, and a < older, > newer legend. Without that check, a typo in a run id gives confusing diff errors.
Corrected check_outputs.sh
bash
#!/bin/bash
check_outputs.sh - re-checks the actual outputs of an earlier test run against
the expected outputs, without running the Front End again.
#
Usage: ./scripts/check_outputs.sh [run folder]
Example: ./scripts/check_outputs.sh (latest run)
./scripts/check_outputs.sh results/run_20261102_2215
#
For every test in the run folder (<feature>/<success|failure>/<test>) it compares:
actual_output.txt with TestCases/<test>/expected_output.txt (termin
actual_daily_transaction.txt with T ases/<test>/daily_transaction.txt (daily
and prints PASS or FAIL, followed by the differences for each failure.
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual


Bash script bug audit
Share
and prints PASS or FAIL, followed by the differences for each failure.
#
Exit code: 0 if every test passes, 1 otherwise (or if no tests are found).
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
TESTS="$ROOT/TestCases"
RUN="${1:-$ROOT/results/latest}"
RUN="${RUN%/}" # drop trailing slash so the prefix s
if [! -d "$RUN"]; then
echo "No run folder found at $RUN (run ./scripts/run_tests.sh first)"
exit 1
fi
shopt -s nullglob
OUTPUTS=("$RUN"/*/*/*/actual_output.txt)
if [${#OUTPUTS[@]} -eq 0]; then
echo "No test outputs found in $RUN"
exit 1
fi
PASS=0
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual


Bash script bug audit
Share
PASS=0
FAIL=0
for ACTUAL_OUTPUT in "${OUTPUTS[@]}"; do
TEST_OUT_DIR="$(dirname "$ACTUAL_OUTPUT")"
TEST_NAME="${TEST_OUT_DIR#"$RUN"/}" # e.g. buy/success/buy01
EXPECTED_DIR="$TESTS/$TEST_NAME"
2>&1 so a missing file shows up as a difference instead of a silent pass
TERMINAL_DIFF="$(diff "$EXPECTED_DIR/expected_output.txt" "$ACTUAL_OUTPUT" 2>&1)"
FILE_DIFF="$(diff "$EXPECTED_DIR/daily_transaction.txt" "$TEST_OUT_DIR/actual_dail
if [-z "$TERMINAL_DIFF"] && [-z "$FILE_DIFF"]; then
PASS=$((PASS + 1))
echo "PASS $TEST_NAME"
else
FAIL=$((FAIL + 1))
echo "FAIL $TEST_NAME"
[-n "$TERMINAL_DIFF"] && printf ' terminal output (< expected, > actual):\n
[-n "$FILE_DIFF"] && printf ' daily transaction file (< expected, > actual)
fi
done
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual


Bash script bug audit
Share
done
echo "----------"
echo "Passed: $PASS Failed: $FAIL Total: $((PASS + FAIL))"
["$FAIL" -eq 0]
Corrected compare_runs.sh
bash
#!/bin/bash
compare_runs.sh - compares the results of two earlier test runs (regression check)
#
Usage: ./scripts/compare_runs.sh <older_run_id> <newer_run_id>
Example: ./scripts/compare_runs.sh run_20261019_140000 run_20261106_101500
Lists tests whose PASS/FAIL status changed and any output that changed between runs.
diff.txt and report.txt files are ignored in the output comparison.
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
if [-z "$1"] || [-z "$2"]; then
echo "Usage: $0 <older_run_id> <newer_run_id>"
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual


Bash script bug audit
Share
echo "Usage: $0 <older_run_id> <newer_run_id>"
echo "Available runs:"; ls "$ROOT/results" 2>/dev/null | grep '^run_'
exit 1
fi
OLD="$ROOT/results/$1"
NEW="$ROOT/results/$2"
for DIR in "$OLD" "$NEW"; do
if [! -f "$DIR/report.txt"]; then
echo "No report.txt found in $DIR"
exit 1
fi
done
echo "Status changes ($1 -> $2, < older, > newer):"
CHANGES="$(diff <(grep -E '^(PASS|FAIL)' "$OLD/report.txt" | awk '{print $2, $1}' | so
<(grep -E '^(PASS|FAIL)' "$NEW/report.txt" | awk '{print $2, $1}' | so
| grep -E '^[<>]')"
if [-n "$CHANGES"]; then echo "$CHANGES" | sed 's/^/ /'; else echo " none"; fi
echo
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual


Bash script bug audit
Share
echo
echo "Output differences between runs:"
DIFFS="$(diff -r -q -x report.txt -x diff.txt "$OLD" "$NEW")"
if [-n "$DIFFS"]; then echo "$DIFFS" | sed 's/^/ /'; else echo " none"; fi
Readability and comment issues
Practice-copy header. Line 2 of both scripts is a "PRACTICE COPY WITH PLANTED BUGS" banner. I removed it from the fixed versions, and it shouldn't be in the real ones.
Example run ids. The two scripts use different formats (run_20261102_2215 vs run_20261106_101500). Use whichever run_tests.sh actually generates in both.
Undocumented exit code. The original check_outputs.sh header never said how the exit code works, which hid the inverted check on line 47. I documented it.
Hard-to-read pass/fail line. The [-n ...] && printf one-liners (lines 40-41) are dense. An if block would read better, but they work as written.
Unquoted $FAIL . Line 47 left $FAIL unquoted, and I quoted it in the fixed version.
Reply
Claude is AI and can make mistakes.
Sonnet 5.5 Medium
Manual